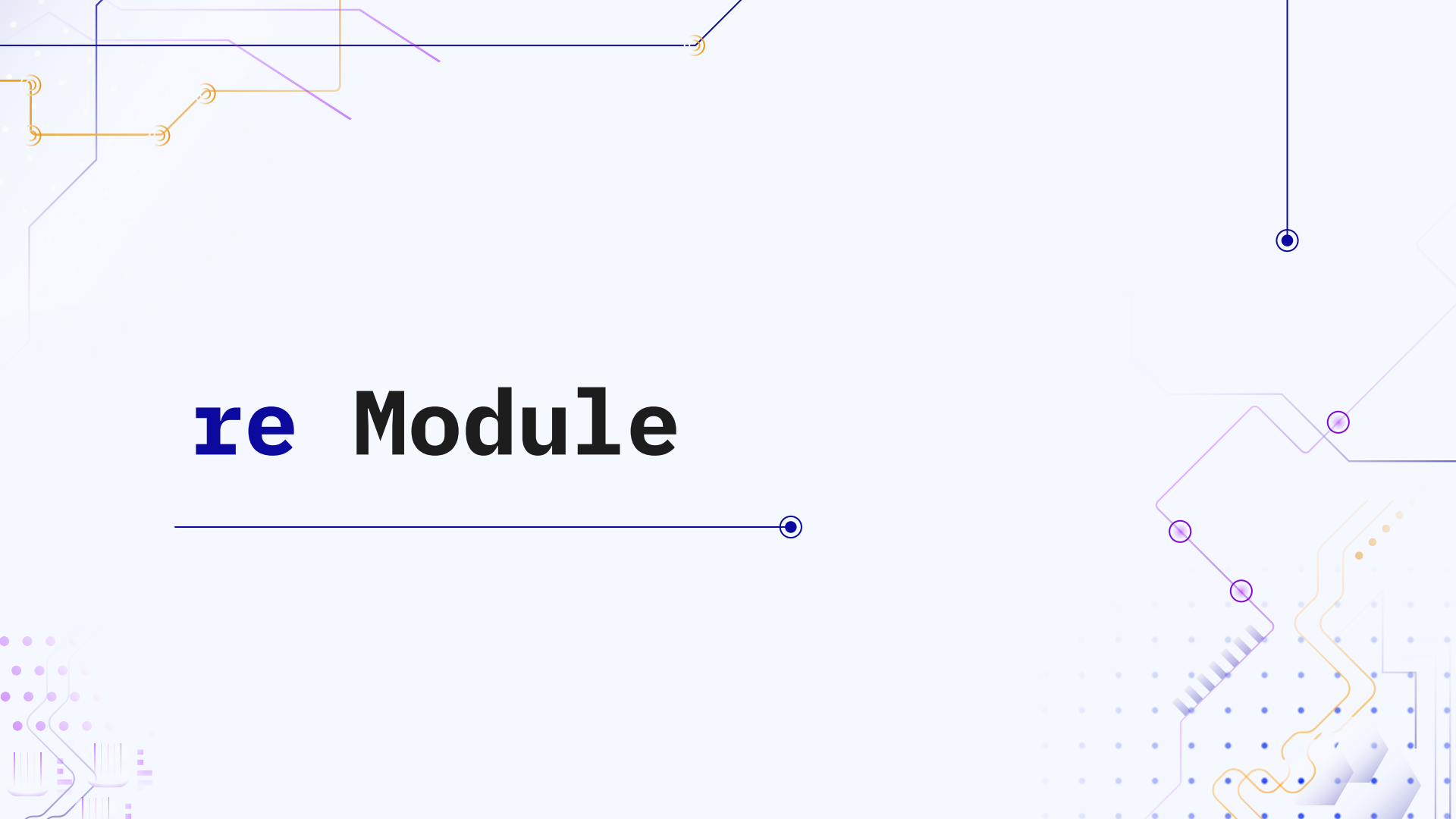




re Module

Introduction

re = regular expressions: a way to find, match, extract, replace, or split text using patterns.

Use it for:

- Validating inputs (email/phone),
- Extracting data from strings,
- Cleaning text,
- Search
- Replace.

Syntax

```
import re
```

Building Blocks

Character classes	
<code>\d</code>	digit (0–9)
<code>\D</code>	not a digit
<code>\w</code>	word char (letters, digits, underscore)
<code>\W</code>	not a word char
<code>\s</code>	whitespace (space/tab/newline)
<code>\S</code>	not whitespace
<code>.</code>	any char except newline (by default)

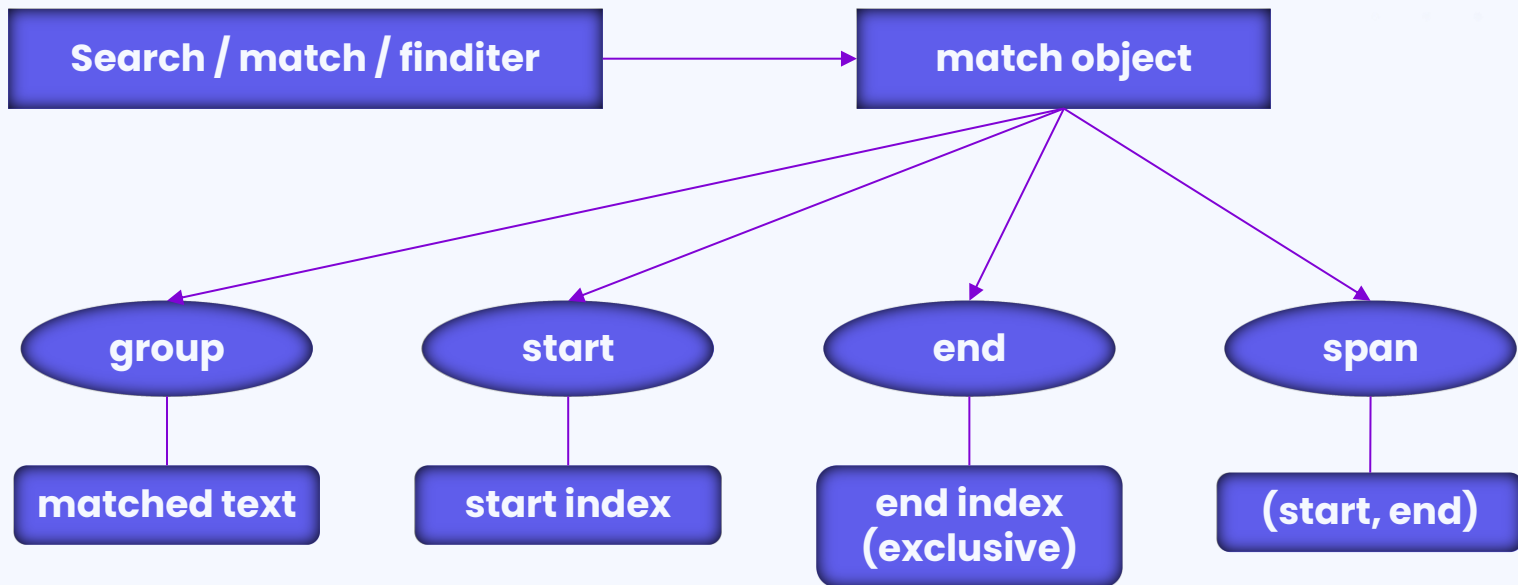
Building Blocks

Custom sets with []	
[abc]	one of a/b/c
[0-9]	any digit
[A-Z]	uppercase
[^0-9]	NOT digit (caret inside [] means negation)
Anchors (position)	
^	start of string
\$	end of string

Building Blocks

Quantifiers (how many times)	
*	0 or more
+	1 or more
?	0 or 1
{n}	exactly n times
{n,}	at least n
{n,m}	between n and m

Most-used re functions



Most-used re functions

A) `re.search(pattern, text):`

Finds **first match anywhere** in the string. Returns a match object or None.

Python

```
m = re.search(r"\d+", "ID: 456")
if m:
    print(m.group())    # 456
```

Most-used re functions

B) `re.match(pattern, text)`

Checks only at the start of the string.

Python

```
re.match(r"\d+", "456 ID") # Match
```

```
re.match(r"\d+", "ID 456") # None
```

Most-used re functions

C) `re.finditer(pattern, text)`

Returns an iterator of match objects for all matches, including their values and positions.

Python

```
for m in re.finditer(r"\d+", "A12 B34"):  
    print(m.group(), m.start(), m.end())
```

Output

```
12 1 3  
34 5 7
```

Most-used re functions

D) `re.findall(pattern, text)`

Returns all matches as a list (strings, or tuples if groups exist).

Python

```
re.findall(r"\d+", "A12 B34 C5") # ['12', '34', '5']
```

Most-used re functions

E) `re.sub(pattern, repl, text)`

Replaces matches.

Python

```
re.sub(r"\s+", " ", "Hi there") # Hi There
```

Most-used re functions

F) `re.split(pattern, text)`

Split string based on pattern.

Python

```
re.split(r"[,;]\s*", "a, b; c") # ['a', 'b', 'c']
```

Quiz Time!!

1:

`r"\d"` matches:

A) Any letter

B) Any digit (0–9)

C) A literal "d"

D) Any whitespace

Quiz Time!!

2:

What does `r"^\d{10}$"` validate

- A) String contains 10 digits anywhere
- B) String starts with 10 digits but can have more
- C) String is exactly 10 digits only
- D) String ends with 10 digits but can have anything before

Quiz Time!!

3:

What does this match: `r"[A-Z]{2}\d{3}"`?

A) 2 lowercase letters + 3 digits

B) 2 uppercase letters + 3 digits

C) 2 uppercase letters + exactly 3 letters

D) Any 2 letters + any 3 digits with spaces allowed

Quiz Time!!

4:

What is the difference between * and + ?

A) No difference

B) * = one or more, + = zero or more

C) * = zero or more, + = one or more

D) * means "exactly 2 times"

Quiz Time!!

5:

What's the output of `re.findall(r"\d+", "A1 B22 C333")` ?

A) ['A1', 'B22', 'C333']

B) ['1', '22', '333']

C) [1, 22, 333]

D) ['A', 'B', 'C']

Quiz Time!!

6:

`r"^(?:\+91[\s-]?|0)?[6-9]\d{9}$"`

Which matches?

A) +91 9876543210

B) 09876543210

C) 9876543210

D) All of the above

Quiz Time!!

7:

`r"^[\w.-]+@[ \w.-]+\.[A-Za-z]{2,}$"`

Which is invalid?

A) a.b-c@x-y.com

B) name@domain.co.in

C) name@domain

D) user_1@abc.org

Quiz Time!!

8:

`r"^[#](?:[0-9a-fA-F]{3}|[0-9a-fA-F]{6})$"`

Which is valid?

A) #12G

B) #abc

C) #abcd

D) #12345

Quiz Time!!

9: Which is invalid?

`r"^https?://[\w.-]+\.[A-Za-z]{2,}(:?/[ \w.-]*)?$"`

A) `https://example.com`

B) `http://abc.co.in/path/to/page`

C) `ftp://example.com`

D) `https://a-b.in/x-y`

WATCH

Level up your coding with each episode in this focused Python series.



Next Video!

PROJECT - 1
DOB Calculator

